



PROJECT REPORT

ENDLESS ZOMBIE

Survival Shooter / Action Roguelite

Information	Details
Semester	Capstone Project
Class	GD1305
Group	The MoonStone
Instructor	Nguyen Dinh Cuong Vu Tran Lam
Group Member	Vu Viet Tien Chu Van Thai

TABLE OF CONTENTS

Contents

I. PROJECT INTRODUCTION	3
1. Project Overview	3
2. Project Scope	3
3. System Name	3
4. Deployment Environment	3
5. Development Tools	3
6. System Features	3
II. SYSTEM REQUIREMENTS ANALYSIS	4
1. System Overview	4
2. Functional Requirements	4
3. Non-functional Requirements	4
4. Use Case Diagram	5
5. Use Case Descriptions	5
Use case 1 – Play game	5
Use Case 2 – Action	6
Use Case 3 – Switch Weapon	7
Use Case 4 – Update Weapon	8
Use Case 5 – Pause & Resume	8
Use Case 6 – Generate & Spawn Enemy Wave	9
Use Case 7 – Auto Attack & Collect EXP, Gold	10
Use Case 8 – Process Enemy Death & Reward	11
6. Activity Diagram	12
III. DETAILED DESIGN	13
1. UI Design	13
2. Hybrid Architecture Diagram	15
3. Class / Component Diagram	15
4. Sequence Diagram	16
6. Data Model	16
Acceptance Criteria	18
V. TASK ASSIGNMENT	19
VII. CONCLUSION AND FUTURE DEVELOPMENT	20

I. PROJECT INTRODUCTION

1. Project Overview

Endless Zombie is a top-down survival shooter in which the player controls an armed survivor inside an arena and attempts to survive increasingly difficult enemy waves. The combat loop combines movement, automatic target acquisition, weapon behavior, experience collection, level-up choices, and persistent upgrades purchased from the main menu.

The project uses a hybrid architecture. GameObject-oriented code handles input, menus, Canvas UI, audio, presentation, and game states. Unity Entities/DOTS handles wave scheduling, entity spawning, movement, projectiles, damage, combat events, and XP. Bridge systems synchronize Entity state with GameObject models, Animator controllers, and visual effects.

2. Project Scope

- A Canvas-based main menu for stage selection, loadout selection, meta upgrades, and settings.
- Player movement, automatic aiming, and several weapon profiles, including a multi-projectile Shotgun.
- Data-driven stage and wave spawning through StageConfig and EnemyCatalog assets.
- Regular zombie and DogMutant visual variants with movement-to-animation speed synchronization.
- A Canvas-based gameplay HUD for health, XP, level, wave status, currency, and weapon statistics.
- Running, paused, level-up, game-over, and win states.
- Local persistence for music, sound, vibration, gold, and meta progression.

3. System Name

Endless Zombie - Hybrid ECS Survival Shooter.

4. Deployment Environment

Item	Configuration
Target platform	Android mobile devices
Engine	Unity 6000.3.10f1
Render pipeline	Universal Render Pipeline 17.3.0
Data-oriented stack	Unity Entities 1.4.2, Entities Graphics 1.4.15, Unity Physics 1.4.2
Language	C#
Input	Unity Input System 1.18.0

5. Development Tools

Tool	Purpose
Unity Editor	Scenes, prefabs, animation, physics, profiling, and builds
Visual Studio / Rider	C# development and debugging
Git	Version control
TextMesh Pro / UGUI	Canvas menus and HUD
ScriptableObject assets	Stage, enemy, weapon, visual, and upgrade configuration

6. System Features

The implemented feature set includes data-driven waves, prefab-based enemy entities, chase behavior, projectiles, damage and status effects, XP and level-up upgrades, weapon profiles, persistent progression, editable Canvas UI, audio controls, and game completion states.

II. SYSTEM REQUIREMENTS ANALYSIS

1. System Overview

The system is organized into four responsibility groups: Presentation, OOP Gameplay, ECS Simulation, and Configuration/Persistence. This division keeps menu and HUD layout editable in the Unity Inspector while moving high-volume gameplay data into cache-friendly ECS systems.

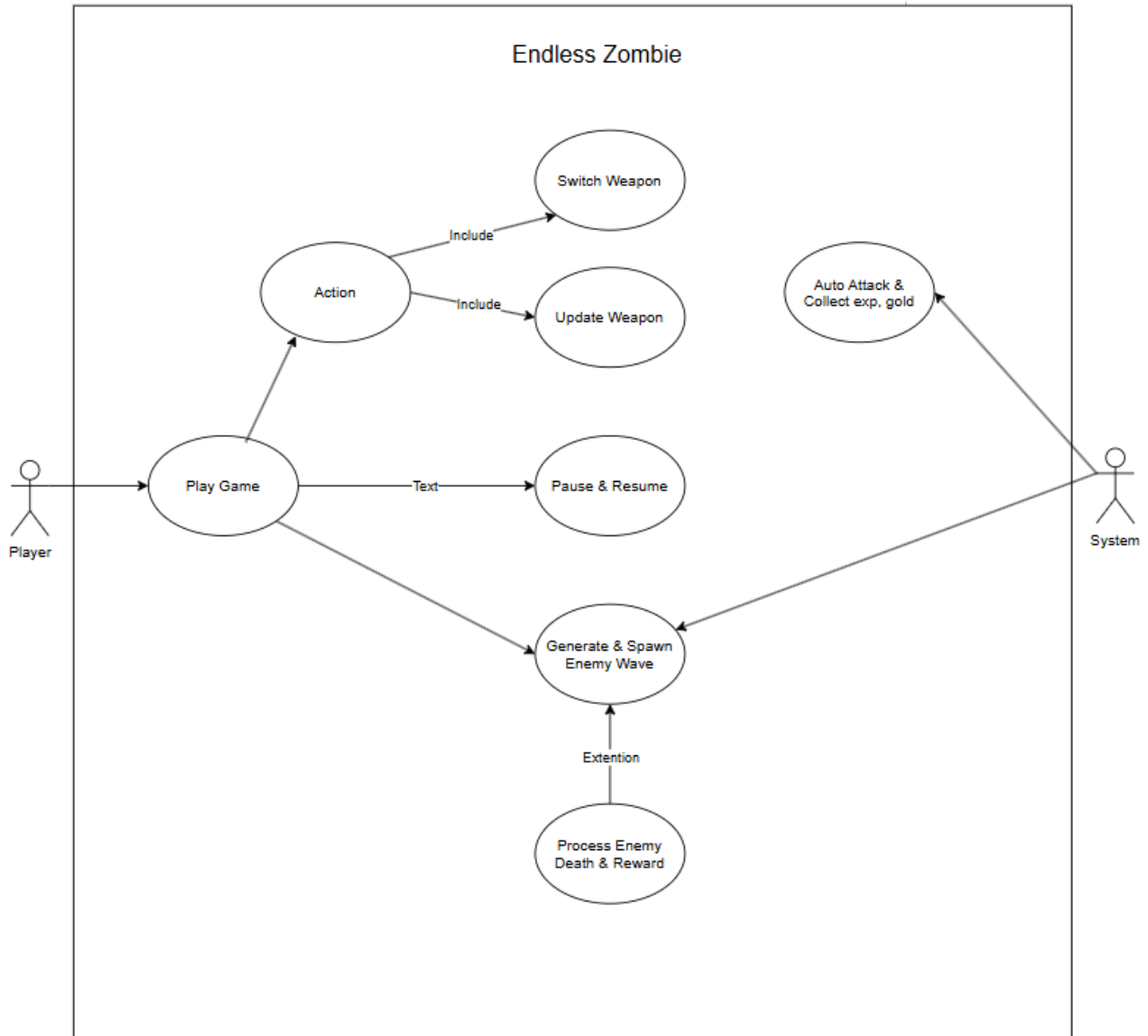
2. Functional Requirements

ID	Requirement	Priority
FR-01	Start a selected stage from the main menu and load GameScene.	Must
FR-02	Automatically acquire a valid target.	Must
FR-03	Spawn enemies by wave, prefab, schedule, and maximum-alive limit.	Must
FR-04	Process projectiles, collisions, damage, death, and XP.	Must
FR-05	Present and apply level-up upgrade choices.	Must
FR-06	Display an Inspector-editable Canvas HUD.	Must
FR-07	Pause, resume, and configure music, sound, and vibration.	Should
FR-08	Persist gold and meta upgrades between sessions.	Should
FR-09	Finish a stage with either Win or Game Over.	Must

3. Non-functional Requirements

- Stable performance with large groups of Entity enemies and limited unnecessary GameObject allocation.
- UI layout must be authored as Canvas prefabs instead of being constructed by runtime layout code.
- Gameplay values must be separated from logic through ScriptableObject configuration.
- Visual run speed must follow Entity movement speed to reduce foot sliding.
- Local settings must have safe defaults when saved values do not exist.
- Wave configuration must validate missing IDs, duplicates, and invalid spawn entries.

4. Use Case Diagram



5. Use Case Descriptions

Use case 1 – Play game

Field	Description
Use Case Name	Play Game
Use Case ID	UC-01
Actor	Player
Organizational Benefits	Allows the player to enter the core gameplay loop and access the main features of Endless Zombie.

Triggers	The player presses the Start button from the Main Menu.
Preconditions	The game has been launched successfully and valid stage, player, and weapon configurations are available.
Postconditions	The gameplay session is initialized and the player enters the active game state.
Main Course	1. Player selects the available stage/loadout.2. Player presses Start.3. System loads GameScene.4. System initializes Player, HUD, weapon data, and ECS runtime.5. System starts the first enemy wave.6. Player enters the gameplay loop.
Alternate Courses	The player may return to the Main Menu or change the selected weapon/loadout before starting the stage.
Exceptions	If required stage or gameplay configuration is missing or invalid, the system reports the error and does not start the gameplay session.

Use Case 2 – Action

Field	Description
Use Case Name	Action
Use Case ID	UC-02
Actor	Player
Organizational Benefits	Enables the player to interact with the gameplay system and control the character during combat.
Triggers	The player provides movement input or performs an available gameplay action.
Preconditions	The gameplay session is active and the game state is Running.
Postconditions	The player's movement or gameplay state is updated according to the performed action.

Main Course	1. Player provides movement input.2. System reads the input.3. System updates player movement and orientation.4. Player continues moving and avoiding enemies.5. Player may access other available actions such as switching or upgrading weapons.
Alternate Courses	The player may remain stationary while the Auto Attack system continues attacking valid targets.
Exceptions	Player actions are ignored when the game is paused, during Level Up selection, or after entering Win/Game Over state.

Use Case 3 – Switch Weapon

Field	Description
Use Case Name	Switch Weapon
Use Case ID	UC-03
Actor	Player
Organizational Benefits	Provides combat variety and allows the player to adapt their attack behavior to different situations.
Triggers	The player selects another available weapon.
Preconditions	The gameplay session is active and more than one valid weapon is available to the player.
Postconditions	The selected weapon becomes the player's active weapon.
Main Course	1. Player opens or uses the weapon selection control.2. Player selects an available weapon.3. System validates the selected weapon.4. System changes the active weapon.5. System applies the selected weapon's current combat statistics.6. Auto Attack continues using the newly selected weapon.
Alternate Courses	The player keeps using the current weapon without making a selection.

Exceptions	If the selected weapon is unavailable or its configuration is invalid, the active weapon remains unchanged.
-------------------	---

Use Case 4 – Update Weapon

Field	Description
Use Case Name	Upgrade Weapon
Use Case ID	UC-04
Actor	Player
Organizational Benefits	Strengthens the player's combat capability and supports roguelite progression during a gameplay session.
Triggers	The player selects a weapon-related upgrade when an upgrade option is available.
Preconditions	The player has reached a valid upgrade opportunity and at least one upgrade option is available.
Postconditions	The selected weapon statistic or weapon modifier is updated.
Main Course	1. Player reaches an upgrade opportunity.2. System displays available upgrade options.3. Player selects a weapon upgrade.4. System validates the selected option.5. System applies the corresponding weapon modifier.6. Updated weapon statistics are used in subsequent attacks.
Alternate Courses	The player may select a non-weapon statistic upgrade if other upgrade types are provided.
Exceptions	If the selected upgrade is invalid or cannot be applied, the current weapon statistics remain unchanged.

Use Case 5 – Pause & Resume

Field	Description
Use Case Name	Pause & Resume

Use Case ID	UC-05
Actor	Player
Organizational Benefits	Allows the player to temporarily stop gameplay and manage settings without losing the current session.
Triggers	The player presses the Pause/Settings button or selects Resume from the pause interface.
Preconditions	A gameplay session is currently active.
Postconditions	The game is either paused or returned to the Running state.
Main Course	1. Player presses Pause.2. System changes from Running to PlayerPaused state.3. Gameplay simulation is suspended.4. Pause/Settings UI is displayed.5. Player presses Resume.6. System restores Running state.7. Gameplay continues.
Alternate Courses	While paused, the player may modify music, sound, or vibration settings before resuming.
Exceptions	Resume cannot restore gameplay if the current session has already entered Win or Game Over state.

Use Case 6 – Generate & Spawn Enemy Wave

Field	Description
Use Case Name	Generate & Spawn Enemy Wave
Use Case ID	UC-06
Actor	Game System
Organizational Benefits	Creates the continuous enemy pressure required for the survival gameplay loop and controls stage difficulty progression.
Triggers	The stage starts or the previous wave has been completed.

Preconditions	A valid StageConfig and wave configuration are available, and the stage is active.
Postconditions	The configured enemies for the wave are spawned and registered as active enemies.
Main Course	1. System activates the current wave.2. System reads the corresponding WaveDefinition.3. Spawn entries are scheduled.4. Spawn requests are generated according to quantity and interval.5. System resolves each EnemyId through EnemyCatalog.6. Enemy Entity prefabs are instantiated.7. Alive enemy count is updated.8. Enemies begin chasing the player.
Alternate Courses	If the maximum-alive-enemy limit is reached, additional spawn requests wait until active enemy count decreases.
Exceptions	Missing, duplicated, or invalid spawn entries are skipped or marked terminal so that wave progression is not permanently blocked.

Use Case 7 – Auto Attack & Collect EXP, Gold

Field	Description
Use Case Name	Auto Attack & Collect EXP, Gold
Use Case ID	UC-07
Actor	Game System
Organizational Benefits	Automates combat interaction, reduces player input complexity, and provides progression rewards through enemy defeats.
Triggers	A valid enemy enters the player's attack range while the game is Running.
Preconditions	The player is alive, the game state is Running, an active weapon is available, and at least one valid enemy is within attack range.
Postconditions	The target may receive damage, and defeating enemies may increase the player's EXP and Gold.

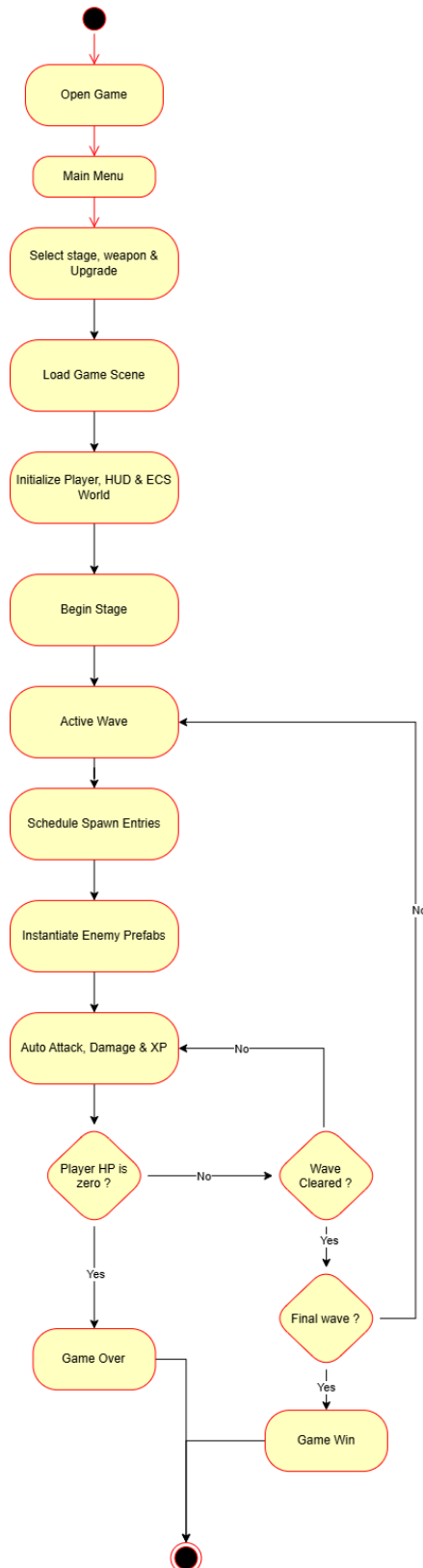
Main Course	1. System searches for valid enemies.2. System selects a valid target within attack range.3. System checks the active weapon's attack cooldown.4. System determines projectile direction and weapon properties.5. System creates projectile(s).6. Projectile collides with an enemy.7. Damage is applied.8. If the enemy dies, the corresponding EXP/Gold reward process is triggered.9. System searches for the next valid target.
Alternate Courses	If the active weapon fires multiple projectiles, the system distributes the projectiles according to the configured spread angle.
Exceptions	If no valid enemy exists within attack range, no attack occurs until another valid target becomes available.

Use Case 8 – Process Enemy Death & Reward

Field	Description
Use Case Name	Process Enemy Death & Reward
Use Case ID	UC-08
Actor	Game System
Organizational Benefits	Ensures defeated enemies are correctly removed, rewards are granted, and wave progression can continue.
Triggers	An enemy's health reaches zero after receiving damage.
Preconditions	An active enemy exists and its health has reached zero.
Postconditions	The enemy is no longer active, its reward is processed, and the current alive-enemy count is reduced.
Main Course	1. System detects that enemy health has reached zero.2. Enemy is marked as defeated.3. Death/combat event is generated.4. EXP and Gold rewards are processed.5. Enemy Entity or visual representation is removed/disabled.6. Alive enemy count is decreased.7. System checks whether the current wave has been completed.8. If the wave is complete, wave progression continues.
Alternate Courses	If other enemies remain alive or spawn entries are still pending, the current wave continues.

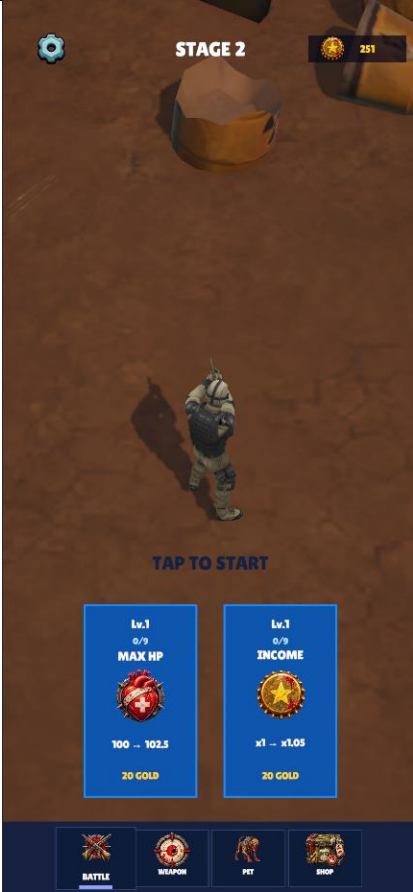
Exceptions	If reward data is unavailable, the enemy is still removed and counted as defeated so that the wave cannot become permanently blocked.
-------------------	---

6. Activity Diagram



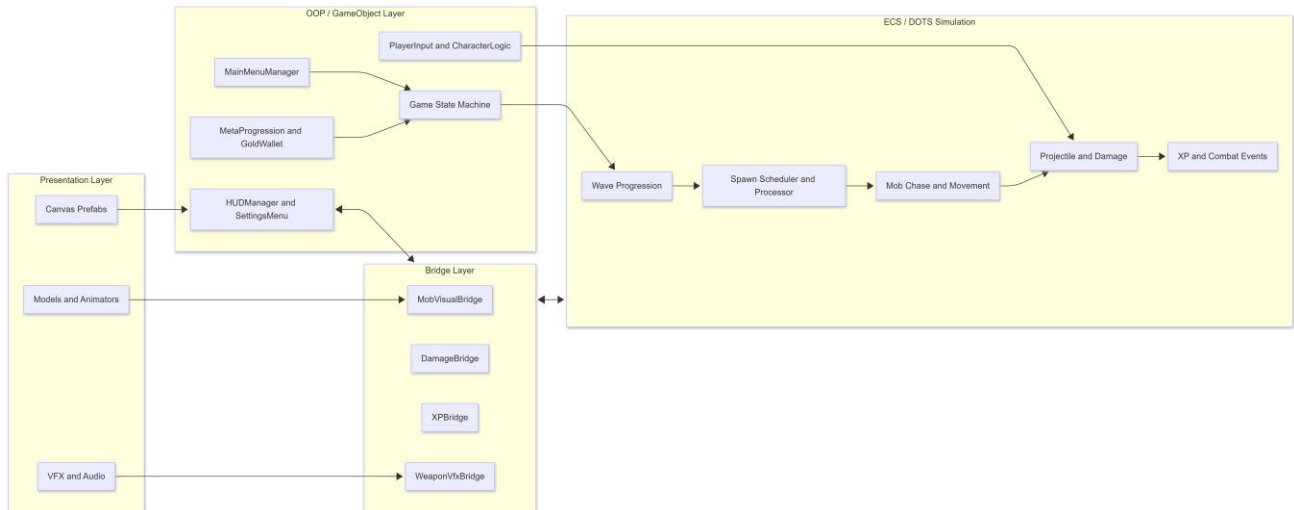
III. DETAILED DESIGN

1. UI Design

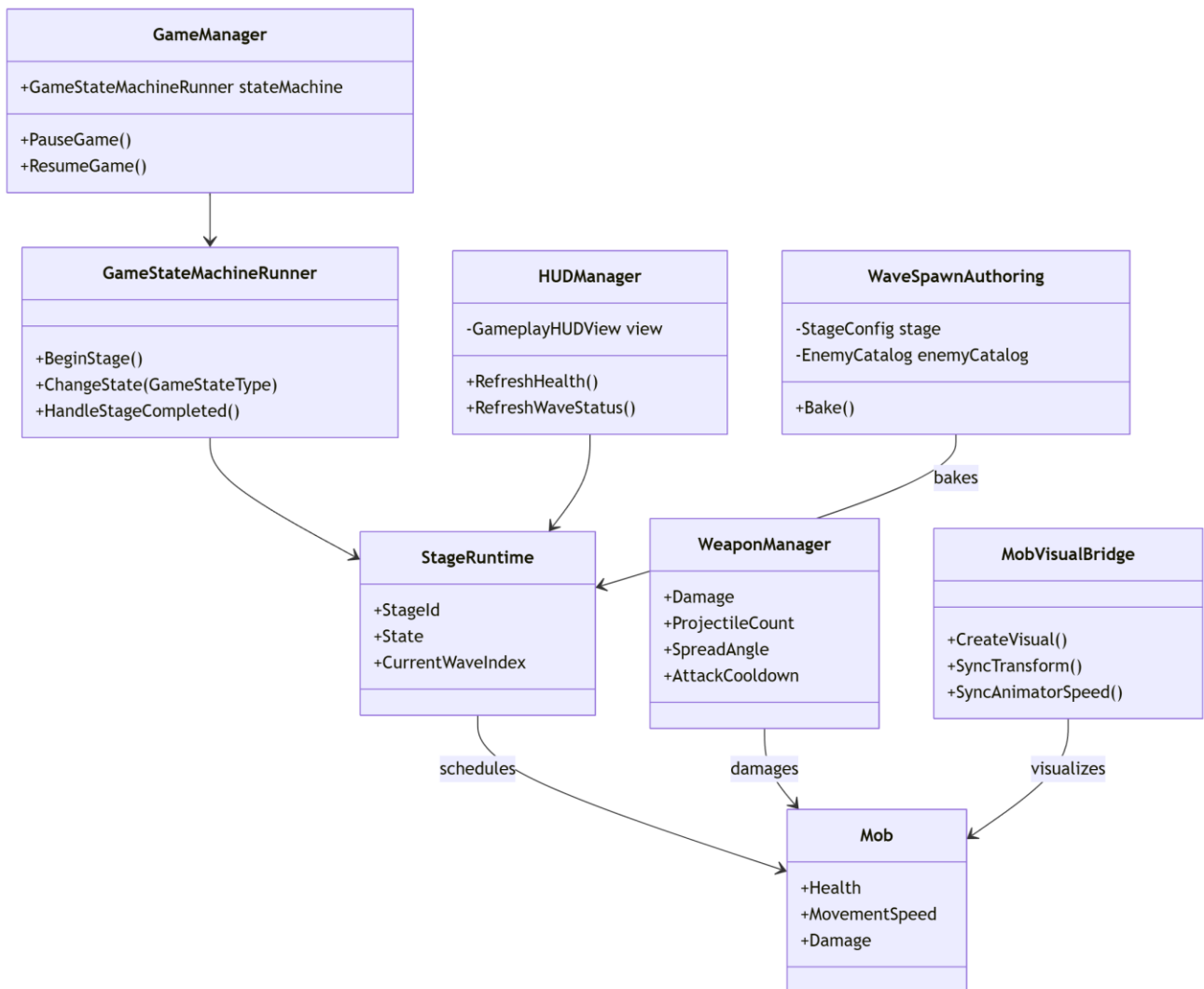
Screen / Prefab	Main elements	Design responsibility
MainMenuCanvas.prefab		MainMenuManager binds data and events; the prefab owns layout.

Screen / Prefab	Main elements	Design responsibility
GameplayHUDLayout.prefab		HUDManager updates values; the Canvas prefab owns layout.
SettingsMenu.prefab		Fill and Handle use matching track padding and remain clamped to the slider bounds.
LevelUpMenu.prefab		Displays upgrade choices while LevelUpState is active.

2. Hybrid Architecture Diagram



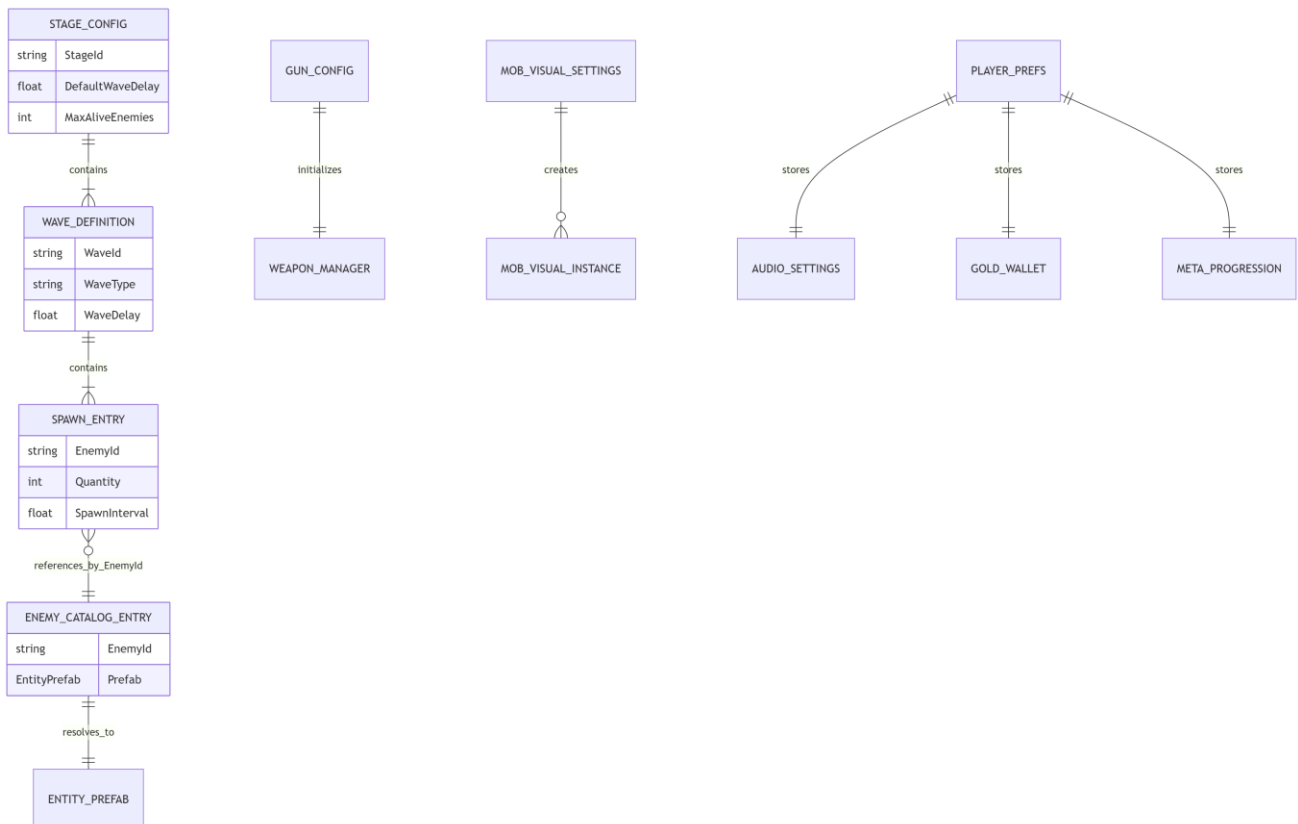
3. Class / Component Diagram



4. Sequence Diagram



6. Data Model



Entity	Representative attributes	Storage
StageConfig	StageId, DefaultWaveDelay, MaxAliveEnemies, Waves	ScriptableObject
WaveDefinition	WaveId, WaveType, ActivationCondition, WaveDelay, Entries	Nested configuration
SpawnEntryDefinition	EnemyId, Quantity, SpawnInterval, spawn position/area	Nested configuration

Entity	Representative attributes	Storage
EnemyCatalogEntry	EnemyId, EntityPrefab	EnemyCatalog.asset
GunConfig	Damage, cooldown, projectile count, spread, range	ScriptableObject
MobVisualSettings	Prefabs, controllers, loop timing, scale/offset data	Resources asset
PlayerPrefs	Music, sound, vibration, gold, meta upgrades	Local machine

IV. TESTING

The table separates static project inspection from Unity Play Mode verification. “Pass - static” confirms that the required code, prefab, or configuration path exists. “Play Mode required” indicates that runtime behavior still needs final execution evidence on the submission machine.

ID	Area	Action	Expected result	Status
TC-01	Main Menu Canvas	Open MainMenu.	The prefab loads and buttons are bound.	Pass - static
TC-02	Gameplay HUD Canvas	Load GameScene.	HUD values are displayed and layout remains prefab-editable.	Pass - static
TC-03	Settings sliders	Drag Music and Sound to both ends.	Fill follows Handle; Handle stays inside track; volume follows value.	Pass - static
TC-04	Player movement	Use movement controls in Running.	The character moves smoothly in the requested direction.	Play Mode required
TC-05	Wave spawning	Begin a stage.	Correct prefabs, quantity, interval, and alive limit are used.	Pass - static
TC-06	Enemy ground contact	Observe spawned enemies.	Collider and visual offset place the model on the ground.	Play Mode required
TC-07	DogMutant animation	Spawn the dog variant.	Run animation plays and speed follows movement speed.	Pass - static
TC-08	Shotgun spread	Equip and fire the Shotgun.	Multiple projectiles form a configured spread cone.	Pass - static
TC-09	XP and level up	Collect enough XP.	Upgrade selection appears and modifies the chosen stat.	Pass - static
TC-10	Pause and resume	Open Settings/Pause and resume.	Simulation pauses and continues in the correct state.	Play Mode required
TC-11	Win condition	Clear the final wave.	StageCompleted is emitted and WinState is entered.	Pass - static
TC-12	Persistence	Change settings and restart.	PlayerPrefs restore the previous values.	Pass - static

Acceptance Criteria

- No compile errors in the Unity Console.
- Enemy models do not flash at the player position during spawn.
- DogMutant shows no obvious foot sliding at default and modified movement speeds.
- Slider fill, handle, and volume value remain synchronized in the [0,1] range.
- The final wave can complete even when an invalid spawn entry has been marked terminal.

V. TASK ASSIGNMENT

No.	Task	Owner	Status
1	Requirement analysis and architecture design	Vu Viet Tien	Completed
2	OOP/ECS gameplay, waves, combat, and XP	Vu Viet Tien Chu Van Thai	Completed for the current version
3	Main Menu, HUD, and Settings Canvas	Chu Van Thai	Completed for the current version
4	Enemy and DogMutant visuals/animation	Chu Van Thai	Completed for the current version
5	Testing, bug fixing, and report writing	Vu Viet Tien	Final Play Mode regression pending

VI. INSTALLATION INSTRUCTIONS

1. Prerequisites

- Unity Hub
- Unity Editor 6000.3.10f1
- Android Build Support module
- Android SDK & NDK Tools
- OpenJDK
- An Android device or Android emulator for testing

2. Open the Project

- Open Unity Hub and choose Add project from disk.
- Select the Endless Zombie project directory.
- Select Unity Editor 6000.3.10f1 and allow package restoration to complete.
- Confirm that the Unity Console contains no compile errors.

3. Run in the Editor

- Open Assets/Scenes/MainMenu.unity.
- Press Play.
- Choose a stage and loadout, then press Tap to Start.
- Verify player movement, HUD, wave spawning, weapons, enemy behavior, pause, settings, and game completion states.

4. Build for Android

- Open File → Build Profiles.
- Select Android as the target platform.
- If Android is not currently active, select Switch Platform.
- Add MainMenu and GameScene to the Scene List.
- Open Player Settings and configure the application package name, orientation, minimum API level, and other Android settings as required.
- Connect an Android device with USB Debugging enabled, or prepare an Android emulator.
- Select Build to generate an APK/AAB, or Build and Run to install and launch the game directly on the connected device.
- Verify that UI scaling, touch input, gameplay, audio, enemy waves, and game states work correctly on the Android device.

VII. CONCLUSION AND FUTURE DEVELOPMENT

Endless Zombie implements the complete survival-shooter loop: stage entry, wave spawning, automatic combat, XP and upgrades, and win/loss completion. The hybrid architecture provides efficient simulation while preserving a visual Unity workflow for UI, models, animation, and VFX.

- Add bosses and more enemy archetypes with explicit attack state machines.
- Introduce pooling for visual bridges and VFX, then profile CPU time and garbage collection.

- Expand stages, weapon synergies, and meta progression.
- Add automated Play Mode tests for spawning, win conditions, and persistence.
- Improve accessibility with input remapping, contrast options, text scaling, and reduced screen shake.